Chapter 4: CRUD Operations with Forms and Requests

Project setup

Now, you will make dedicated pages for the CRUD operations. Create a file structure that looks like this:

```
your_project/
├── app.py
└── templates/
    ├── home.html
    ├── add_user.html
    └── view_user.html
```

Install the required packages:

```
pip install flask flask-sqlalchemy pymysql cryptography
```

Form pages

Copy the following code snippets into their respective HTML files. They only introduce HTML tables and forms as the input and output interfaces.

Home page

This page lists all the users in the database. The buttons direct visitors to the other pages.

User Management

Add New User

ID	Username	Email	Action
1	testuser	test@user.com	Manage
2	testuser2	test@user2.com	Manage

templates/home.html

```
<!DOCTYPE html>
<html>
<head>
    <title>User Management</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
crossorigin="anonymous">
</head>
<body>
    <div class="container mt-5">
        <h1>User Management</h1>
        <!-- Flash messages -->
        {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
                {% for category, message in messages %}
                <div class="alert alert-{{ category }}">{{ message }}</div>
                {% endfor %}
            {% endif %}
        {% endwith %}
        <a href="{{ url_for('add_user') }}" class="btn btn-primary mb-3">Add New
User</a>
```

templates/home.html (continued)

```
        <table class="table">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Username</th>
                    <th>Email</th>
                    <th>Action</th>
                </tr>
            </thead>
            <tbody>
                {% for user in users %}
                <tr>
                    <td>{{ user.id }}</td>
                    <td>{{ user.username }}</td>
                    <td>{{ user.email }}</td>
                    <td>
                        <a href="{{ url_for('view_user', user_id=user.id) }}"
class="btn btn-secondary btn-sm">Manage</a>
                    </td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
crossorigin="anonymous"></script>
</body>
</html>
```

Add-user page

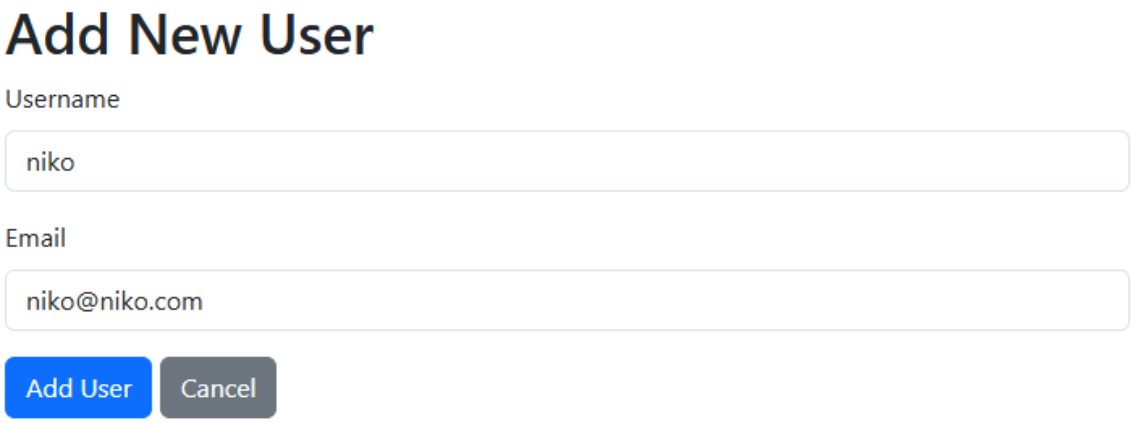
This page introduces a form element with the submit method set as "POST". Carefully read through it and make sure you understand every single line.

templates/add_user.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Add User</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
crossorigin="anonymous">
</head>
<body>
    <div class="container mt-5">
        <h1>Add New User</h1>
        {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
        {% for category, message in messages %}
        <div class="alert alert-{{ category }}">{{ message }}</div>
        {% endfor %}
        {% endif %}
        {% endwith %}
        <form method="POST">
            <div class="mb-3">
                <label for="username" class="form-label">Username</label>
                <input type="text" class="form-control" id="username"
name="username" required>
            </div>
            <div class="mb-3">
                <label for="email" class="form-label">Email</label>
                <input type="email" class="form-control" id="email" name="email"
required>
            </div>
            <button type="submit" class="btn btn-primary">Add User</button>
            <a href="{{ url_for('index') }}" class="btn btn-secondary">Cancel</a>
        </form>
    </div>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
crossorigin="anonymous"></script>
</body>
</html>
```

- `<form method="POST">` creates a native HTML form element that sends all fields within it to the same URL (in this case `/add`) with the method `POST` instead of the regular `GET`.
- `<input>` elements contain the variables. The variable names are set by the `name` attribute.Add `type` and `required` attributes where necessary.

- `<button type="submit">` then makes the POST API request, transmitting the filled out form fields.



View-user page

This page mixes the concepts of the home and add_user pages: It has the same form elements but pre-fills them with the data from the database. The buttons at the bottom now need a specific `formaction` attribute because they lead to different routes than the one that serves this page. They specify the url and the parameter

templates/view_user.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Manage User</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
crossorigin="anonymous">
</head>
<body>
    <div class="container mt-5">
        <h1>Manage User</h1>
        {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
        {% for category, message in messages %}
        <div class="alert alert-{{ category }}">{{ message }}</div>
        {% endfor %}
        {% endif %}
        {% endwith %}
```

templates/view_user.html (continued)

```
        <form method="POST">
            <div class="mb-3">
                <label for="username" class="form-label">Username</label>
                <input type="text" class="form-control" id="username"
name="username" value="{{ user.username }}" required>
            </div>
            <div class="mb-3">
                <label for="email" class="form-label">Email</label>
                <input type="email" class="form-control" id="email" name="email"
value="{{ user.email }}" required>
            </div>
            <button
                type="submit"
                formaction="{{ url_for('update_user', user_id=user.id) }}"
                class="btn btn-primary"
            >
                Update
            </button>

            <button
                type="submit"
                formaction="{{ url_for('delete_user', user_id=user.id) }}"
                class="btn btn-danger"
                onclick="return confirm('Are you sure?')"
            >
                Delete
            </button>

            <a
                href="{{ url_for('index') }}"
                class="btn btn-secondary"
            >
                Cancel
            </a>
        </form>
    </div>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
crossorigin="anonymous"></script>
</body>
</html>
```

Updates to the server code

You need to make changes to the server code of the previous chapter. Primarily, *whenever an action makes changes to the database,* you no longer pass new information as URL parameters (e.g., `/add_user/<str:username>/`). Instead, you now use **POST requests** and transmit the new variables in the request body. Only use GET for safe read-only operations.

Routes will now look like this:

```
@app.route('/add', methods=['GET', 'POST']) ❶
def add_user():
    if request.method == 'POST': ❷
        username = request.form['username'] ❸
        email = request.form['email']
        if not username or not email:  ❹
            flash('Please fill in all fields', 'error')
            return redirect(url_for('add_user'))
        try:
            new_user = User(username=username, email=email)
            db.session.add(new_user)
            db.session.commit()
            flash('User added successfully!', 'success')
            return redirect(url_for('index')) ❺
        except Exception as e:
            flash(f'Error adding user: {str(e)}', 'error')
            return redirect(url_for('add_user')) ❻

    return render_template('add_user.html') ❼
```

❶ The route now accepts both "GET" and "POST" as a method.

❷ The database manipulation part of the function is only executed if the request method is "POST".

❸ This extracts what the user typed into the form field with the label "username".

4. If one of the fields is missing, the server shows an error message and reloads the page to show it.
5. If the new user has been added successfully, the server redirects the browser to the landing page.
6. If there was an exception when adding the user, the server shows a different error message and reloads the page to show it.
7. If the method is "GET", this function simply renders the template.

The sever still accepts GET requests

- `GET` (e.g., just typing in the URL) reads data and renders the template.
- `POST` (i.e., form action) makes database changes.
- In real-world systems you would use more specific HTTP methods like `PATCH`, `PUT`, and `DELETE`. To keep it simple, stick to `POST` for this class.

Make sure you understand every line of the `add_user()` function and then insert it into the template for the server code below. Note that you have to import `request` from the Flask package for this. Then implement the missing edit and delete functions. Use the same logic you learned in the previous chapter.

app.py

```
from flask import Flask, render_template, redirect, url_for, flash, request
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

with app.app_context():
    db.create_all()
```

app.py (continued)

```python
@app.route('/')
def index():
    users = User.query.all()
    return render_template('home.html', users=users)

# CREATE - Add new user
@app.route('/add', methods=['GET', 'POST'])
def add_user():

    # TODO: Implement add user functionality

    return render_template('add_user.html')

# READ - View an individual user
@app.route('/view/<int:user_id>', methods=['GET'])
def view_user(user_id):
    user = User.query.get_or_404(user_id)
    return render_template('view_user.html', user=user)

# UPDATE - Edit an individual user
@app.route('/update/<int:user_id>', methods=['POST'])
def update_user(user_id):
    user = User.query.get_or_404(user_id)

    # TODO: Implement update user functionality

    return redirect(url_for('view_user', user_id=user_id))

# DELETE - Delete a user
@app.route('/delete/<int:user_id>', methods=['POST'])
def delete_user(user_id):

    # TODO: Implement delete user functionality

    return redirect(url_for('index'))
```

Test your implementation by adding, editing, and deleting users!